Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval)

An Engineering Standard and Operational Procedure for Neural Network Optimization Algorithms

Silver Moon (`cloverxion`)

Lumina Moon (`cloverx-id`)

October 2026

Engineering Standard Specification — Rev 1.0.0

Abstract

This manual defines the official **Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval)** synchronized with the **LuminaMoon v1.0.0** optimization suite. It establishes a repeatable, four-stage testing protocol to evaluate new stochastic optimization algorithms, hardware execution backends (Rust AVX-512/AVX2 SIMD, Vulkan 1.3/1.4 Wave32 UMA, Triton AOT/JIT, and zero-copy NumPy/PyTorch frontends), and version updates before production deployment. LSO-Eval replaces ad-hoc trial-and-error with a structured four-stage filtering pipeline: (I) Formal Logic, Rational Padé Approximation, and Bitwise Precision Verification, (II) Synthetic Stress-Test Surfaces, Cross-Backend Numerical Parity, and State Serialization Integrity, (III) Short-Run Proxy Model Screening, and (IV) Large-Scale Pre-Training Evaluation. This specification includes checklists, standard test harnesses, acceptance criteria, and version-regression gates.

---

1 Scope, Principles, and the Staged Evaluation Pipeline

1.1 Objective and Operational Scope

The LSO-Eval standard governs the qualification of all first-order and quasi-second-order optimization algorithms and hardware acceleration kernels developed or deployed within the Lumina Moon research and production infrastructure. The objective is four-fold:

1. **Prevent Compute Wastage:** Filter out mathematically flawed, numerically unstable, or overly sensitive candidates at near-zero compute cost before allocating GPU clusters.
2. **Separate Implementation Bugs from Theory:** Isolate algebraic flaws, rational function approximation errors, low-precision stochastic rounding bugs, and hardware kernel overhead into distinct test phases.
3. **Guarantee Cross-Backend Consistency:** Enforce strict numerical parity across the Scalar Reference Implementation (`luminav-core`), x86_64 AVX-512/AVX2 SIMD (`luminav-simd`), Vulkan 1.3/1.4 UMA Wave32 compute shaders (`luminav-vulkan`), Triton AOT/JIT GPU kernels (`luminav-triton-aot`), and NumPy/PyTorch bindings (`PyO3 abi3/abi3t`).

4. **Enforce Non-Regression:** Ensure that algorithmic and systems updates improve upon or preserve prior baselines across key metrics (loss, memory footprint, and wall-clock latency).

1.2 Staged Filtering Protocol

Candidate algorithms must pass through four sequential stages. Failure at any stage halts the evaluation immediately; engineering teams must not proceed to large-scale training while lower-stage checks remain unverified.

Table 1: The LSO-Eval Staged Evaluation Pipeline (v1.0.0)

Stage	Designation	Core Verification Objective	Cost Ceiling
Stage I	Math & Bitwise Check	Zero division-by-zero, unit velocity bounds, subnormal floor safety, [13/6] Padé tanh accuracy, zero stochastic rounding bias.	$\mathcal{O}$(ms)
Stage II	Synthetic & Parity	Ill-conditioned traversal ($\kappa \geq 1000$), saddle escape, λ^* balance, 1-Pass EMA fusion, cross-backend numerical parity, 6-slot scratch & checkpoint integrity.	$\mathcal{O}$(sec)
Stage III	Short-Run Proxy	Matrix update-to-weight ratio stability ($R_t \leq 0.10$), early loss descent, learning rate & bound mode stability mapping.	$\mathcal{O}$(min)
Stage IV	Large-Scale Run	Low-precision FP16/BF16 stability, FSDP collective reduction parity, peak VRAM reduction, kernel throughput, held-out perplexity.	$\mathcal{O}$(hours)
D-Eval	Version Regression	4-Gate comparative audit evaluating release candidate v_k against baseline predecessor v_{k-1}.	Cross-Stage

2 Stage I: Formal Logic & Symbolic Verification Protocol

Stage I requires formal verification of algebraic safety properties and bitwise precision bounds before running any tensor training loop. The candidate optimizer is expressed in symbolic real arithmetic and checked using automated theorem provers and statistical equivalence tests.

2.1 Standard SMT Safety Checklist (Z3 Solver)

The engineering team formulates the optimizer update velocity $u_t(\theta, g, m, v)$ and checks the following seven safety properties in the Z3 Theorem Prover [1]. Each property is verified by proving that its **negation** is `UNSAT` (unsatisfiable):

Q1. Denominator Non-Singularity (Division-by-Zero): For any gradient $g \in \mathbb{R}$, momentum $m \in \mathbb{R}$, and centered or uncentered defect residual $d_t = (g - m) - \mathbb{I}_{\text{cd}}\mu_d$, prove that the envelope denominator $\mathcal{E}$ strictly satisfies $\mathcal{E} > 0$. Negated query: $\mathcal{E} \leq 0$.

Q2. Velocity Regularity & Profile Boundedness: Prove that the update velocity satisfies its design profile:

- *Profile A (Coordinate-Bounded / Normalized, e.g., LuminaV, Lion [2]):* Strictly bounded magnitude $|u_t| < 1.0$. Negated query: $|u_t| \geq 1.0$.
- *Profile B (Variance-Adaptive, e.g., AdamW [3], RMSprop):* Denominator strictly positive $\sqrt{v_t}+\epsilon > 0$ and non-divergent quotient as $g \to 0$. Negated query: $\sqrt{v_t}+\epsilon \leq 0$.
- *Profile C (Matrix-Preconditioned / Quasi-Second-Order, e.g., Muon, Shampoo [4]):* Spectral/operator norm boundedness $\|U_t\|_2 \leq C$ and non-singular preconditioning $\sigma_{\min}(P) > 0$.

Q3. Radial Scaling & Cautious Rescaling Bounds: For any radial scaling factor $S(r)$, prove that $0 < S(r) \leq 1.0$ for all $r \geq 0$. For cautious masking with density floor $c_{\min} \in (0, 1]$, prove that the rescaling multiplier $\gamma_{\text{cautious}} = 1/\text{clamp}(\bar{m}, c_{\min}, 1.0)$ satisfies $\gamma_{\text{cautious}} \in [1.0, 1/c_{\min}]$.

Q4. Subnormal Numerical Floor: For half-precision formats (FP16), prove that adaptive safety bounds and effective epsilons remain strictly above the FP16 subnormal threshold (6.1035×10^{-5}): $\epsilon_{\text{eff}} = \max(\epsilon, 10^{-4})$ and $b_{\text{safe}} = \max(0.01|\theta_0|, 1.5 \times 10^{-4})$. Negated query: $b_{\text{safe}} \leq 6.1 \times 10^{-5} \vee \epsilon_{\text{eff}} \leq 6.1 \times 10^{-5}$.

Q5. Descent Direction Guarantee: Prove that under zero initial momentum ($m_0 = 0$) or cautious alignment masking ($m_{\text{mask}} = \mathbb{I}(u_t \cdot g_t > 0)$), the effective update preserves non-negative alignment with the gradient: $\langle \delta_t, g_t \rangle \geq 0$.

Q6. Weight Decay Contraction: For base learning rate $\eta \in (0, 0.05]$ and decoupled weight decay [3] $\lambda_{\text{wd}} \in [0, 0.2]$, prove that $(1 - \eta\lambda_{\text{wd}}) \in (0, 1]$. Negated query: $(1 - \eta\lambda_{\text{wd}}) \leq 0 \vee (1 - \eta\lambda_{\text{wd}}) > 1$.

Q7. Lookahead Bias Correction Positivity: For all integer steps $t \geq 1$ and $\beta_1, \beta_2 \in (0, 1)$, prove that the Nesterov lookahead bias correction $\text{bc}_{1,\text{nes}} = 1 - \beta_1^{t+1}$ and clamped second-moment correction $\text{bc}_2 = \max(1 - \beta_2^t, 10^{-15})$ remain strictly positive. Negated query: $\text{bc}_{1,\text{nes}} \leq 0 \vee \text{bc}_2 \leq 0$.

2.2 Standard Z3 SMT Verification Harness

Listing 1 provides the executable Stage I test script verifying both Single-Buffer TD-CLM and Dual-Buffer Belief-CLM properties, including centered defect shifts and cautious rescaling bounds.

2.3 Symbolic Calculus, Rational Padé Audit & Bitwise Precision

After passing SMT verification, the optimizer undergoes four analytical and bitwise precision checks:

S1. Large-Gradient Limit Check: Using SymPy [5], evaluate the symbolic limit of velocity u_t as gradient magnitude approaches infinity:

$$u_\infty = \lim_{|g| \to \infty} u_t(m, g). \tag{1}$$

Listing 1: Standard LSO-Eval Stage I: Z3 SMT Verification Harness (v1.0.0)

```python
from z3 import *

def audit_optimizer_safety(candidate_name="LuminaV-v1.0.0"):
    solver = Solver()
    m, g, mu_d, v = Reals('m g mu_d v')
    w1, w2, lam, eps, eps_fp16, b_safe = Reals('w1 w2 lam eps eps_fp16 b_safe')
    m_bar, c_min = Reals('m_bar c_min')

    preconds = [
        w1 > 0, w2 > 0, (w1 + w2) == 1, lam >= 0, eps > 0,
        eps_fp16 >= 1e-4, b_safe >= 1.5e-4, v >= 0,
        c_min == 0.5, m_bar >= c_min, m_bar <= 1.0
    ]

    abs_m = If(m >= 0, m, -m)
    abs_g = If(g >= 0, g, -g)
    diff = (g - m) - mu_d
    abs_diff = If(diff >= 0, diff, -diff)

    candidate_num = w1 * m + w2 * g
    candidate_den = w1 * abs_m + w2 * abs_g + lam * abs_diff + eps
    abs_num = If(candidate_num >= 0, candidate_num, -candidate_num)
    cautious_scale = 1.0 / m_bar

    solver.push()
    solver.add(preconds, candidate_den <= 0)
    res_div0 = solver.check()
    solver.pop()

    solver.push()
    solver.add(preconds, abs_num >= candidate_den)
    res_bound = solver.check()
    solver.pop()

    solver.push()
    solver.add(preconds, Or(b_safe <= 6.1e-5, eps_fp16 <= 6.1e-5))
    res_subnormal = solver.check()
    solver.pop()

    solver.push()
    solver.add(preconds, Or(cautious_scale < 1.0, cautious_scale > (1.0 / c_min)))
    res_cautious = solver.check()
    solver.pop()

    checks = [
        ("Division-by-Zero Prevention", res_div0),
        ("Unconditional Unit Boundedness", res_bound),
        ("FP16 Subnormal Floor Safety", res_subnormal),
        ("Cautious Scale Boundedness [1, 2]", res_cautious),
    ]

    all_passed = True
    for name, res in checks:
        status = "PASSED" if res == unsat else "FAILED"
        if res != unsat:
            all_passed = False
        print(f"[{candidate_name}] {name:<35}: {status}")
    return all_passed
```

Acceptance Rule: For Profile A (Coordinate-Bounded), the limit must remain strictly below unit magnitude ($|u_\infty| < 1.0$). For Profile B and C, the limit must remain finite without numerical overflow ($\lim_{|g|\to\infty}\|u_t\| < \infty$).

S2. Piecewise Continuity (C^0/C^1 Transition): When radial bounding switches from unity scaling ($S(r) = 1.0$ for $r < r_{\text{thresh}}$) to $\tanh(r)/r$ at threshold $r_{\text{thresh}} = 10^{-4}$, compute the transition step:

$$\Delta_{\text{jump}} = \left|1.0 - \frac{\tanh(r_{\text{thresh}})}{r_{\text{thresh}}}\right| \approx \frac{r_{\text{thresh}}^2}{3}. \tag{2}$$

Acceptance Rule: Δ_{jump} must be strictly smaller than single-precision machine epsilon ($\epsilon_{\text{FP32}} \approx 1.19 \times 10^{-7}$). At $r_{\text{thresh}} = 10^{-4}$, $\Delta_{\text{jump}} \approx 3.33 \times 10^{-9} \ll \epsilon_{\text{FP32}}$.

S3. Rational Padé [13/6] Approximation Accuracy: For hardware backends without native single-cycle tanh instructions (x86_64 AVX-512/AVX2 and Vulkan SPIR-V), $\tanh(x)$ is evaluated via the clamped [13/6] rational Padé polynomial on $x \in [-7.9988117, 7.9988117]$:

$$\tanh_{\text{Padé}}(x) = \frac{x\sum_{k=0}^{6}\alpha_{2k+1}x^{2k}}{\sum_{j=0}^{3}\beta_{2j}x^{2j}}. \tag{3}$$

Acceptance Rule: Maximum absolute error across $\mathbb{R}$ against IEEE 754 `std::f32::tanh` must satisfy $\|\tanh_{\text{Padé}}(x) - \tanh(x)\|_\infty < 1.5 \times 10^{-6}$, while preserving odd symmetry $\tanh_{\text{Padé}}(-x) = -\tanh_{\text{Padé}}(x)$ and NaN propagation.

S4. Stochastic Rounding Unbiasedness & MurmurHash3 Determinism: Low-precision quantization (`bfloat16` and `float16`) uses the deterministic MurmurHash3 32-bit finalizer seeded per parameter and step:

$$\text{seed}_m = (\text{seed}_0 + t \cdot \texttt{0x9E3779B9} + \text{idx}_{\text{param}} \cdot 10007)\ \&\ \texttt{0x7FFFFFFF}, \tag{4}$$

$$\text{seed}_p = (\text{seed}_m + \texttt{0x55555555})\ \&\ \texttt{0x7FFFFFFF}. \tag{5}$$

For FP16, the bitmask width adapts to subnormal exponents ($\exp_{\text{f32}} < 113$) via $\text{shift}_{\text{total}} = 13 + \text{clamp}(113 - \exp_{\text{f32}}, 0, 11)$. Generate $N = 200{,}000$ samples across both normal ($[10^{-3}, 10^{-1}]$) and FP16 subnormal ($[10^{-5}, 6.1 \times 10^{-5}]$) ranges. Evaluate unbiasedness via a two-sided Student's t-test ($H_0 : E[\mathcal{Q}_{\text{SR}}(x)] = x$) and Two One-Sided Tests (TOST) within tolerance $\pm\delta_{\text{tol}} = 10^{-6}$. **Acceptance Rule:** Mean bias must satisfy $|\mu_{\mathcal{Q}} - x| < 10^{-6}$ ($p_{\text{TOST}} < 0.01$, or t-test $p \geq 0.01$) across both normal and subnormal ranges.

Reference Case Example: LuminaV v1.0.0 TD-CLM & Dual-Buffer Verification in Stage I

In LuminaV v1.0.0, first moment state updates via $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$. Single-Buffer TD-CLM (`buffer=1`) and Dual-Buffer Belief-CLM (`buffer=2`) compute raw update velocity via:

$$u_t^{(\text{B1})} = \frac{w_1 m_{t-1} + w_2 g_t}{w_1|m_{t-1}| + w_2|g_t| + \lambda|d_t^{(\text{B1})}| + \epsilon_{\text{eff}}}, \quad w_1 = \beta_1^2, \quad w_2 = 1 - \beta_1^2,$$

$$u_t^{(\text{B2})} = \tanh\left(\frac{\beta_1 m_t + (1-\beta_1)g_t}{\max(c_1\sqrt{\max(v_t, 0)} + c_2, 10^{-12})}\right), \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)\left(d_t^{(\text{B2})}\right)^2,$$

where the centered defect residuals and lookahead bias-correction scalars are defined as:

$$d_t^{(\mathrm{B1})} = (g_t - m_{t-1}) - \mathbb{I}_{\mathrm{cd}}\mu_d, \quad d_t^{(\mathrm{B2})} = (g_t - m_t) - \mathbb{I}_{\mathrm{cd}}\beta_1\mu_d, \quad \mu_d = \frac{1}{N}\sum_{i=1}^{N}(g_{t,i} - m_{t-1,i}),$$

$$c_1 = \frac{(1-\beta_1^{t+1})\tau}{\sqrt{\max(1-\beta_2^t, 10^{-15})}}, \quad c_2 = \epsilon_{\mathrm{eff}}(1-\beta_1^{t+1})\tau.$$

Note the algebraic identity $\beta_1 m_t + (1-\beta_1)g_t \equiv \beta_1^2 m_{t-1} + (1-\beta_1^2)g_t = w_1 m_{t-1} + w_2 g_t$, unifying the Nesterov lookahead numerator across both buffer modes.

1. **Z3 SMT Result:** By the triangle inequality, $|w_1 m_{t-1} + w_2 g_t| \le w_1|m_{t-1}| + w_2|g_t|$. Because $\lambda|d_t^{(\mathrm{B1})}| \ge 0$ and $\epsilon_{\mathrm{eff}} > 0$, the Buffer 1 denominator strictly exceeds the numerator, while Buffer 2 satisfies $|\tanh(\cdot)| < 1.0$ with denominator floor $10^{-12} > 0$, returning `UNSAT` on $|u_t| \ge 1.0$.
2. **Large-Gradient Ceiling:** SymPy limit analysis on Buffer 1 yields $u_\infty = \frac{w_2}{w_2+\lambda}$. For standard defaults ($\beta_1 = 0.9 \implies w_2 = 0.19$, $\lambda = 0.30$), velocity during extreme gradient spikes is capped at $u_\infty = \frac{0.19}{0.49} \approx 0.3878 < 1.0$.
3. **Stochastic Rounding & State Quantization:** Monte Carlo evaluation over 200k samples of MurmurHash3 bitwise rounding (with subnormal exponent-shift support for FP16) yielded mean bias 1.68×10^{-7} ($p = 0.48$, $p_{\mathrm{TOST}} < 0.001$). First moment m_t uses stochastic rounding, whereas second moment v_t uses deterministic round-to-nearest-even (RNE) with an FP16 ceiling of 65,000.0 to prevent variance overflow.

3 Stage II: Synthetic Test Surfaces, Cross-Backend Parity & State Serialization

Stage II connects mathematical verification (Stage I) to neural network training. Its goal is to test optimizer trajectories on challenging synthetic surfaces, calibrate default hyperparameters, verify numerical consistency across hardware backends, and check state serialization integrity.

3.1 Synthetic Benchmark Functions

The engineering team evaluates the candidate optimizer against reference baselines (standard AdamW [3] and the previous version) across five standard test functions [8]:

B1. Ill-Conditioned Quadratic Bowl ($\kappa \ge 1000$):

$$f(w_0, w_1) = \frac{1}{2}\left(w_0^2 + 1000 w_1^2\right), \quad w^{(0)} = [10.0, 1.0]^\top. \tag{6}$$

Evaluation Focus: Tests whether the triangle-defect envelope or belief variance dampens steep-wall oscillations along w_1 without slowing progress along the flat direction w_0.

B2. Non-Convex Rosenbrock Curved Valley [8, 9]:

$$f(w_0, w_1) = (1 - w_0)^2 + 100(w_1 - w_0^2)^2, \quad w^{(0)} = [-1.5, 2.0]^\top. \tag{7}$$

Evaluation Focus: Tests the optimizer's ability to follow curved non-linear valleys where the gradient direction shifts continuously.

B3. Stochastic Gradient Noise Resilience: Evaluates the quadratic bowl under additive Gaussian gradient noise $\tilde{g}_t = \nabla f(w_t) + \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 1.0$. **Evaluation Focus:** Measures steady-state error and noise amplification near the optimum.

B4. Multimodal Rastrigin Local-Minima Escape [10]:

$$f(w_0, w_1) = 20 + \sum_{i=0}^{1} \left(w_i^2 - 10\cos(2\pi w_i) \right), \quad w^{(0)} = [2.5, 2.5]^\top. \tag{8}$$

Evaluation Focus: Evaluates whether lookahead momentum carries the iterate out of shallow local minima toward the global minimum at $(0, 0)$.

B5. High-Dimensional Saddle Point Escape ($N \geq 100$):

$$f(w) = \frac{1}{2} \sum_{i=1}^{N} \lambda_i w_i^2 + \frac{1}{4} \sum_{i=1}^{N} w_i^4, \quad \lambda_i \in [-1.0, 1000.0]. \tag{9}$$

Evaluation Focus: Evaluates the optimizer's ability to escape high-dimensional saddle points (negative curvature eigenvalues) and verifies coordinate/radial bounding stability.

3.2 Standard 2D Simulation Harness

Listing 2 provides the deterministic Python harness for Stage II trajectory testing, implementing the LuminaV v1.0.0 TD-CLM update step with cautious rescaling and coordinate bounding.

3.3 Hyperparameter Sensitivity & Equilibrium Calibration

Optimizers often fail in practice when their performance depends on a narrow hyperparameter window. LSO-Eval requires systematic sweeps across all hyperparameters:

P1. One-at-a-Time Sweep Protocol: Fix standard defaults ($\beta_1 = 0.9, \beta_2 = 0.999, \tau = 0.8, \epsilon = 10^{-8}$) and vary the target hyperparameter across at least 5 values spanning an order of magnitude.

P2. Analytical Balance Point: Whenever a parameter controls a trade-off (such as spike damping vs. sign-reversal responsiveness), derive the analytical balance point in closed form and enforce input validation bounds in code.

P3. Parameter Stability Range: The default value must sit within a stable range where convergence speed degrades by no more than 15% under a ±30% perturbation.

3.4 Feature Combinations, 1-Pass EMA Parity & Cross-Backend Numerical Parity

Because LuminaMoon v1.0.0 combines multiple algorithmic options and hardware execution backends, Stage II enforces three parity checks:

Listing 2: Standard LSO-Eval Stage II: Deterministic Simulation Harness (v1.0.0)

```python
import numpy as np

def run_2d_benchmark(loss_fn, grad_fn, step_fn, w_init, f_star=0.0,
                     lr=0.01, max_steps=2000, tol=1e-5, noise_std=0.0, seed=1337):
    rng = np.random.default_rng(seed)
    w = np.array(w_init, dtype=np.float64)
    state = {}
    current_loss = loss_fn(w)

    for t in range(1, max_steps + 1):
        g = grad_fn(w)
        if noise_std > 0.0:
            g = g + rng.normal(0.0, noise_std, size=g.shape)

        w, state = step_fn(w, g, state, lr, t)
        current_loss = loss_fn(w)

        if np.isnan(w).any() or np.isinf(w).any() or (current_loss - f_star) > 1e8:
            return t, current_loss, "DIVERGED"
        if abs(current_loss - f_star) < tol:
            return t, current_loss, "CONVERGED"

    return max_steps, current_loss, "MAX_STEPS"

def luminav_tdclm_step_fn(w, g, state, lr, t,
                          beta1=0.9, lam=0.3, eps=1e-8, bound_ratio=0.1):
    if "m" not in state:
        state["m"] = np.zeros_like(w)

    m = state["m"]
    w1 = beta1 * beta1
    w2 = 1.0 - w1

    m_new = beta1 * m + (1.0 - beta1) * g
    clm_m = w1 * m + w2 * g
    diff = g - m
    clm_env = w1 * np.abs(m) + w2 * np.abs(g) + lam * np.abs(diff) + eps

    u = clm_m / clm_env
    mask = (u * g > 0.0).astype(np.float64)
    cautious_scale = 1.0 / np.clip(np.mean(mask), 0.5, 1.0)

    step_val = lr * (u * mask) * cautious_scale
    b_i = bound_ratio * np.sqrt(w * w + 1e-4)
    step_val = b_i * np.tanh(step_val / b_i)

    state["m"] = m_new
    w_next = w - step_val
    return w_next, state
```

A1. Feature Combination Stability: Across all combinations of $\mathcal{C} = \mathcal{M}_{\text{buffer}} \times \mathcal{M}_{\text{bound}} \times \mathcal{M}_{\text{cautious}} \times \mathcal{M}_{\text{centered_defect}}$, enabling any subset of features must remain numerically stable. Coordinate bounding applies per-parameter soft saturation:

$$\Delta\theta_i = B_i \tanh\left(\frac{\eta\delta_i}{B_i}\right), \quad B_i = r_{\text{bound}}\sqrt{\theta_{0,i}^2 + 10^{-4}}, \tag{10}$$

while radial bounding applies global norm scaling $S(r) = \tanh(r)/r$ for $r = \frac{\eta\|\delta\|_2}{r_{\text{bound}}\max(\|\theta_0\|_2, 1.0)}$.

A2. 1-Pass Fused Scratchpad EMA Parity: In 1-Pass Fused mode (`fused_single_pass=True`, active for step $t > 1$), each parameter maintains a 6-element `float32` scratchpad vector:

$$\mathbf{s} = [\underbrace{\Sigma m_{\text{mask}}}_{s_0}, \underbrace{\Sigma u_{\text{eff}}^2}_{s_1}, \underbrace{\Sigma\theta_0^2}_{s_2}, \underbrace{\gamma_{\text{cautious}}}_{s_3}, \underbrace{S_{\text{bound}}}_{s_4}, \underbrace{\Sigma(g-m)}_{s_5}] \in \mathbb{R}^6. \tag{11}$$

The kernel reads s_3 and s_4 from step $t-1$ while accumulating s_0, s_1, s_2 in a single memory pass, then updates s_3, s_4 post-step via EMA with mode-specific defaults (`ema_decay="auto"` $\implies \alpha_{\text{ema}} = 0.95$ for `buffer=1` and $\alpha_{\text{ema}} = 0.80$ for `buffer=2`). The final loss difference between 1-Pass Fused EMA and 2-Pass Instantaneous execution must not exceed 1.0%.

A3. Cross-Backend Numerical Parity (`luminav-core`, `luminav-simd`, `luminav-vulkan`):

- *Parallel Chunking & Pairwise Tree Reduction:* Multithreaded Rayon execution partitions tensors into chunks of `ELEM_CHUNK_SIZE` = 2,097,152 elements using 16-lane pairwise tree reduction (`reduce_sum_16`). It must achieve exact bitwise parity (`atol` = 0.0) against single-threaded scalar execution.
- *Bitwise PRNG & Stochastic Rounding Parity:* AVX-512 and AVX2 vectorized PRNG and FP16/BF16 stochastic rounding must achieve exact `0.0` bit-level parity against the scalar reference implementation.
- *Multi-Step Trajectory Parity (N = 131,072, 5 steps):* Across both TD-CLM and Dual-Buffer modes in FP32 and BF16, AVX-512, AVX2, and Vulkan 1.3/1.4 Wave32 UMA compute shaders must match the Scalar Reference within atol $\leq 10^{-5}$ (FP32) and atol $\leq 2 \times 10^{-2}$ (BF16).
- *Frontend Wrapper Parity:* Standalone NumPy (`StandaloneOptimizer` via zero-copy `__array_interface__`) and PyTorch (`torch.optim.Optimizer`) must produce identical parameter trajectories on matching inputs.

3.5 State Cleanup & Checkpoint Serialization Audit

To prevent silent resume bugs or memory leaks, the optimizer must pass the **LSO-Eval Serialization Audit** (Listing 3), verifying: (1) no temporary contiguous staging buffers remain in the state dictionary, (2) temporary reduction accumulators (s_0, s_1, s_2, s_5) are reset to zero while EMA scales (s_3, s_4) remain in `float32`, (3) `master_param` retains `float32` precision, and (4) parameter updates after reloading match an uninterrupted run bit-for-bit.

Listing 3: Standard LSO-Eval Stage II: Checkpoint Roundtrip & State Cleanup Test (v1.0.0)

```python
import copy, io, torch

def verify_checkpoint_integrity(optimizer_cls, model_fn):
    torch.manual_seed(1337)
    model_cont = model_fn().to(dtype=torch.bfloat16)
    model_res = copy.deepcopy(model_cont)

    opt_cont = optimizer_cls(model_cont.parameters(), master_weights="fp32")
    opt_res = optimizer_cls(model_res.parameters(), master_weights="fp32")
    batches = [torch.randn(4, 10, dtype=torch.bfloat16) for _ in range(6)]

    for i in range(5):
        for m, opt in ((model_cont, opt_cont), (model_res, opt_res)):
            opt.zero_grad()
            m(batches[i]).sum().backward()
            opt.step()

    buffer = io.BytesIO()
    torch.save(opt_res.state_dict(), buffer)
    buffer.seek(0)

    opt_loaded = optimizer_cls(model_res.parameters(), master_weights="fp32")
    state_dict = torch.load(buffer, weights_only=False)
    opt_loaded.load_state_dict(state_dict)

    forbidden_keys = (
        "p_contig", "exp_avg_contig", "exp_avg_sq_contig",
        "mp_contig", "_plan", "_contig_step"
    )
    for p_state in state_dict["state"].values():
        for k in forbidden_keys:
            assert k not in p_state, f"Memory leak: '{k}' in state_dict!"
        if "scratch" in p_state:
            sc = p_state["scratch"]
            assert float(sc[0]) == 0.0 and float(sc[1]) == 0.0
            assert float(sc[2]) == 0.0 and float(sc[5]) == 0.0

    for p in model_res.parameters():
        st = opt_loaded.state[p]
        if "scratch" in st:
            assert st["scratch"].dtype == torch.float32
        if "master_param" in st:
            assert st["master_param"].dtype == torch.float32

    for m, opt in ((model_cont, opt_cont), (model_res, opt_loaded)):
        opt.zero_grad()
        m(batches[5]).sum().backward()
        opt.step()

    for p_c, p_r in zip(model_cont.parameters(), model_res.parameters()):
        torch.testing.assert_close(p_c, p_r, rtol=0.0, atol=0.0)
    return True
```

Reference Case Example: LuminaV v1.0.0 Trajectory, Cross-Backend Parity & Serialization in Stage II

Proposition 1 (Symmetric Spike–Reversal Balance for λ)**.** *Let $w_1 = \beta_1^2$ and $w_2 = 1 - \beta_1^2$ with $\beta_1 \in (\frac{\sqrt{3}}{2}, 1)$. In TD-CLM, the velocity ceiling under a large gradient spike ($|g_t| \to \infty$) is $u_{spike}(\lambda) = \frac{w_2}{w_2+\lambda}$, and the instantaneous velocity ceiling under an exact gradient sign reversal ($g_t = -m_{t-1}$) is $u_{rev}(\lambda) = \frac{w_1-w_2}{1+2\lambda}$. The unique coefficient λ^* that equalizes spike damping and reversal damping ($u_{spike}(\lambda^*) = u_{rev}(\lambda^*)$) is given in closed form by:*

$$\lambda^* = \frac{2w_2^2}{w_1 - 3w_2} = \frac{2(1-\beta_1^2)^2}{4\beta_1^2 - 3}, \quad \forall \beta_1 \in \left(\frac{\sqrt{3}}{2}, 1\right). \tag{12}$$

Proof. Setting $\frac{w_2}{w_2+\lambda} = \frac{w_1-w_2}{1+2\lambda}$ yields $w_2(1+2\lambda) = (w_1 - w_2)(w_2 + \lambda)$. Expanding and collecting terms in λ gives $w_2(1 - w_1 + w_2) = (w_1 - 3w_2)\lambda$. Substituting $1 - w_1 = w_2$ produces $2w_2^2 = (w_1 - 3w_2)\lambda$, which solves directly to $\lambda^* = \frac{2(1-\beta_1^2)^2}{4\beta_1^2-3}$ for $4\beta_1^2 - 3 > 0 \iff \beta_1 > \frac{\sqrt{3}}{2} \approx 0.866$. Substituting standard $\beta_1 = 0.9$ yields $\lambda^* \approx 0.3008 \approx 0.30$. □

LuminaV v1.0.0 was verified across all Stage II checks:

- **2D Traversal ($\kappa = 1000$):** Unregularized B-CLM ($\lambda = 0.0$) oscillated between the w_1 walls and failed to reach 10^{-5} loss within 1,000 steps. Adding TD-CLM ($\lambda = 0.30$) damped transverse oscillations and converged smoothly in 521 steps.
- **Balance Calibration (λ^*):** Proposition 1 derived the closed-form balance point $\lambda^* \approx 0.3008$, matching the spike ceiling (0.3878) and reversal ceiling (0.3875).
- **Cross-Backend Numerical Parity:** All 7 parity test suites in `luminav-simd` and 5 UMA shader test suites in `luminav-vulkan` matched the `luminav-core` scalar reference across 131,072-element tensors over 5 steps (FP32 and BF16).
- **Serialization Roundtrip:** Passing Listing 3 confirmed that temporary staging buffers and transient accumulators (s_0, s_1, s_2, s_5) are cleared while preserving `float32` precision on s_3, s_4 and `master_param`.

4 Stage III: Short-Run Proxy Model Screening

Stage III tests candidate configurations on smaller neural networks before committing to full training runs. By running short training schedules on proxy models, teams can filter out unstable settings and map effective learning rate ranges early.

4.1 Proxy Model Selection & Step Budget

The engineering team selects a smaller proxy model representative of the target architecture (e.g., a 100M–500M parameter Transformer or ConvNet):

M1. Step Limit: The screening run is capped at 5% to 15% of the full training budget (typically 500 to 2,000 optimization steps).

M2. Batch Size Diversity: Evaluate at least two batch sizes (standard micro-batch and 4× accumulated batch) to check update stability under different gradient noise levels.

M3. Controlled Schedule: Use an identical linear warmup (5% of steps) followed by cosine decay across all candidates.

4.2 Early Stopping & Rejection Criteria

A candidate run is stopped and rejected immediately if any of the following issues occur:

F1. Update-to-Weight Ratio Breach: Compute the layer-wise relative update norm across weight matrices (ndim ≥ 2, excluding 1D biases and normalization scales):

$$R_{t,l} = \frac{\|\delta_{t,l}\|_2}{\|\theta_{t,l}\|_2}. \tag{13}$$

Rejection Rule: If $R_{t,l} > 0.10$ for any matrix layer l (excessive step size) or $R_{t,l} < 10^{-7}$ during active training ($t > t_{\text{warmup}}$ and $\eta_t \geq 0.1\eta_{\max}$, indicating stalled updates), the configuration is rejected.

F2. Unrecovered Loss Spikes: If the exponential moving average (EMA) training loss jumps by more than 0.5 units and fails to return to its pre-spike level within 50 steps, the run is rejected.

F3. Low-Precision Stalling: If the loss curve flattens early while FP32 baselines continue decreasing, the update rule suffers from low-precision mantissa truncation [7].

4.3 Learning Rate & Bounding Mode Grid

Because bounded update velocities interact with the base learning rate, engineers evaluate a learning rate grid ($\eta \in [8 \times 10^{-4}, 4 \times 10^{-2}]$) paired with bounding modes (`bound_type="coordinate"` vs. `"radial"`):

- **Long-Run Setting ($>$ 20k steps):** Identify conservative learning rates (default $\eta = 8 \times 10^{-4}$, $\lambda_{\text{wd}} = 8 \times 10^{-2}$) where single-buffer and dual-buffer modes converge similarly.
- **Short-Run Setting ($<$ 10k steps):** Identify higher learning rates (5× to 20× larger) kept stable by coordinate-wise hyperbolic bounding.

Reference Case Example: LuminaV v1.0.0 Proxy Screening in Stage III

During Stage III screening of LuminaV v1.0.0:

- **Step-Budget Recommendations:** At $\eta = 8 \times 10^{-4}$, Buffer 1 (TD-CLM) and Buffer 2 (Dual-Buffer CLM) matched in loss descent, establishing $\eta = 8 \times 10^{-4}$ as the shared default. For short runs ($<$ 10k steps), raising η to 0.01–0.02 with `bound_type="coordinate"` sped up early descent by 20× without violating $R_{t,l}$ bounds.
- **Default Bounding Mode:** Coordinate bounding (`bound_type="coordinate"`, `bound_ratio=0.1`) handled heavy gradient noise better than radial bounding, since per-element saturation prevents individual outlier coordinates from scaling down the entire layer's update.

5 Stage IV: Large-Scale Pre-Training Evaluation

Stage IV is the final empirical validation phase, divided into two testing tiers:

1. **Stage IV-A (Hardware Qualification, Auto-Router & Burn-in Check):** Medium-length runs (2,000 to 5,000 steps, context length 512–2,048) measuring VRAM usage, kernel latency, hardware auto-router efficiency (`GPU_DISPATCH_THRESHOLD` = 65,536 elements routing small 1D biases to AVX-512/AVX2 and large matrices to Vulkan UMA / GPU), FSDP collective sharding parity, and early convergence.
2. **Stage IV-B (Full-Schedule & Long-Context Evaluation):** Full training runs ($>$ 20,000 steps, context length $\geq$ 2,048/8,192) checking late-stage learning rate decay stability, gradient spike behavior, and held-out validation perplexity across the full training schedule.

5.1 Hardware Profiling & Distributed Sharding Protocol

To ensure a fair comparison, all candidate and baseline optimizers run on identical hardware nodes with matched random seeds, data loaders, and sequence lengths. When testing under PyTorch Fully Sharded Data Parallel (FSDP) with `fsdp_sync=True`, local reduction values $[\Sigma m_{\text{mask}}, \Sigma u_{\text{eff}}^2, \Sigma \theta_0^2, N_{\text{local}}]$ are packed into a single asynchronous `all_reduce` call per parameter group to avoid communication overhead. The engineering team records four core metrics (Table 2):

Table 2: Stage IV: Standard Evaluation Metrics

Dimension	Metric	Measurement Method & Tooling
1. Convergence	Raw Loss & EMA Loss	Recorded at every step; smoothed via EMA ($\alpha = 0.99$).
2. Wall-Clock	Elapsed Time vs. Loss	Total elapsed time in seconds from step 1 to completion.
3. Memory	Peak VRAM Allocated (GB)	Measured via `torch.cuda.max_memory_allocated()` [12].
4. Latency	Optimizer Kernel Time (ms)	Measured via hardware timers / CUDA events around `opt.step()`.

5.2 Generalization & Held-Out Perplexity Check

Fast training loss reduction is only useful if the model generalizes well. Stage IV requires evaluating model checkpoints on held-out validation datasets:

$$\text{PPL}_{\text{held-out}} = \exp\left(\frac{1}{N}\sum_{i=1}^{N}\mathcal{L}_{\text{CE}}(x_i)\right). \tag{14}$$

Acceptance Rule: The candidate optimizer must achieve held-out validation perplexity equal to or lower than the reference AdamW baseline [3] ($\Delta\text{PPL} \leq 0.0$) at the same token count.

Reference Case Example: LuminaV v1.0.0 4B-Parameter LLM Benchmark in Stage IV-A (Hardware Qualification)

LuminaV v1.0.0 was evaluated on the **Qwen 4B Base** model (2,500 steps, sequence length 512, native `bfloat16` parameters):

Optimizer Configuration	Final EMA Loss	Peak VRAM	Step Time	Opt Kernel	Total Time
LuminaV (Buffer 1: TD-CLM)	**6.9194**	**26.60** GB	**186.7** ms	**31.6** ms	**466.7** s
LuminaV (Buffer 2: Dual)	7.0798	34.55 GB	194.1 ms	42.5 ms	485.4 s
AdamW (Native BF16) [3]	7.8351	34.55 GB	192.6 ms	40.9 ms	481.5 s
AdamW (FP32 Master Weights) [3]	6.9985	72.28 GB	276.8 ms	123.3 ms	692.0 s

Key Results:

1. **Eliminating BF16 Stalling:** Native BF16 AdamW stalled at loss 7.8351 due to mantissa truncation [7]. LuminaV's in-kernel MurmurHash3 stochastic rounding [6, 7] prevented underflow without FP32 master weights, reaching 6.9194.
2. **Memory Savings:** LuminaV Buffer 1 reduced peak VRAM by 45.68 GB (63.2% lower) compared to FP32 Master AdamW and 7.95 GB (23.0% lower) compared to Native BF16 AdamW.
3. **Step Throughput:** Fused single-pass execution [13] reduced optimizer step time to 31.6 ms (3.9× faster than FP32 Master AdamW), cutting total training time by 32.6%.

6 Differential Versioning & Regression Protocol (D-Eval)

When updating an existing optimizer or backend kernel from version v_{k-1} to v_k, LSO-Eval requires the **D-Eval Protocol** to ensure no silent performance or numerical regressions are introduced.

6.1 The Four Anti-Regression Gates

A release candidate must pass all four gates before deployment (Table 3):

6.2 Performance Delta Comparison

For Gate 4, the engineering team reports the performance differences against the previous version:

$$\Delta\mathcal{L}_{\text{EMA}} = \mathcal{L}_{\text{EMA}}(v_k) - \mathcal{L}_{\text{EMA}}(v_{k-1}) \leq 0, \tag{15}$$

$$\Delta\text{VRAM}_{\text{peak}} = \text{VRAM}(v_k) - \text{VRAM}(v_{k-1}) \leq 0, \tag{16}$$

$$\Delta T_{\text{kernel}} = T_{\text{kernel}}(v_k) - T_{\text{kernel}}(v_{k-1}) \leq 0. \tag{17}$$

If any metric regresses ($\Delta > 0$), the team must document the trade-off in an Architecture Decision Record (ADR) explaining why the change is justified.

Table 3: The D-Eval Four-Gate Anti-Regression Framework

Gate	Evaluation Target	Acceptance Criterion	Verification Tool
Gate 1	Algebraic Safety	All Stage I SMT queries return `UNSAT`; Padé & SR bounds preserved.	Z3 [1] / SymPy [5]
Gate 2	Trajectory & Backend Parity	Head-to-head 2D run on $\kappa \geq 1000$ and cross-backend parity tests pass.	2D Sim & Rust Tests
Gate 3	Checkpoint Compatibility	Checkpoints saved in version v_{k-1} load cleanly into v_k with identical dtypes and trajectories.	State-Dict Audit
Gate 4	Production Metrics Delta	Meets non-regression bounds: $\Delta\mathcal{L} \leq 0$, $\Delta\text{VRAM} \leq 0$, $\Delta T_{\text{kernel}} \leq 0$.	4B Scale Benchmark

Reference Case Example: D-Eval Audit: LuminaV v1.0.0 Release vs. Prototype Baseline

Summary of the LuminaMoon v1.0.0 release audit across Rust, Vulkan, Triton, and Python:

- **Gate 1 (SMT & Calculus):** Unified Convex Lookahead Momentum weights ($w_1 = \beta_1^2, w_2 = 1 - \beta_1^2$) across Buffer 1 and Buffer 2 ($w_1 + w_2 = 1$) and verified [13/6] Padé tanh accuracy ($< 1.5 \times 10^{-6}$ max error).
- **Gate 2 (Trajectory & Backend Parity):** Verified TD-CLM ($\lambda = 0.30$) spike damping ($u_{\text{spike}} \approx 0.3878$) and passed all numerical parity tests across Scalar, AVX2, AVX-512, Rayon parallel chunking, Vulkan 1.3/1.4 Wave32 UMA, and Triton AOT/JIT.
- **Gate 3 (Migration & State Cleanup):** Reset 6-slot `float32` scratchpad accumulators (zeroing indices $0, 1, 2, 5$ on export), removed temporary contiguous staging keys, and added Levenshtein typo suggestions for configuration keys.
- **Gate 4 (Efficiency):** Maintained $\Delta\text{VRAM} = 0$ GB (26.60 GB on 4B LLM) and packaged zero-dependency sub-1MB CPython Stable ABI (`abi3-py310` / `abi3t`) wheels.

7 Standard LSO-Eval Certification Checklist

No optimizer or hardware kernel update may be merged into main production branches until every item in Table 4 is verified and signed off.

Table 4: Official LSO-Eval v1.0.0 Release Certification Checklist

Stage	ID	Verification Target	Acceptance Criterion	Sign-off
Stage I	I.1	SMT Safety Suite	All negated safety queries return `UNSAT`.	[ ]
	I.2	Large-Gradient Limit	Profile A: $\lim_{\|g\|\to\infty} \|u_t\| < 1.0$; Profile B/C: $\lim_{\|g\|\to\infty} \|u_t\| < \infty$.	[ ]
	I.3	Padé tanh & Continuity	$\|\tanh_{\text{Padé}} - \tanh\|_\infty < 1.5 \times 10^{-6}$; radial jump $\Delta_{\text{jump}} < \epsilon_{\text{FP32}}$.	[ ]
	I.4	Stochastic Rounding	Unbiased rounding ($\|\mu_{\mathcal{Q}} - x\| < 10^{-6}$, $p_{\text{TOST}} < 0.01$ or t-test $p \geq 0.01$) over 200k samples.	[ ]
Stage II	II.1	Synthetic Test Surfaces	Stable convergence on $\kappa \geq 1000$, Rosenbrock, Rastrigin, saddle, and noisy functions.	[ ]
	II.2	Balance Calibration	Default $\lambda = 0.30$ sits in a stable range; λ^* symmetry verified.	[ ]
	II.3	1-Pass EMA Parity	Fused 1-Pass matches 2-Pass loss within 1.0% ($\alpha_{\text{ema}} \in \{0.95, 0.80\}$).	[ ]
	II.4	Cross-Backend Parity	Scalar, AVX2, AVX-512, Vulkan Wave32 UMA, Triton, and NumPy/PyTorch parity verified.	[ ]
	II.5	Checkpoint Roundtrip	No staging keys; $s_{0,1,2,5}$ zeroed; FP32 state dtypes & trajectory preserved on reload.	[ ]
Stage III	III.1	Update Norm Stability	Matrix (ndim ≥ 2) active-phase update ratio satisfies $R_{t,l} \in [10^{-7}, 0.10]$.	[ ]
	III.2	Loss Descent Stability	Zero unrecovered loss spikes over the short proxy run.	[ ]
Stage IV	IV-A.1	Hardware & FSDP Parity	Peak/step VRAM, kernel latency, auto-router threshold, and FSDP sharding meet targets.	[ ]
	IV-A.2	Early Convergence (IV-A)	Initial loss descent matches or beats reference AdamW.	[ ]
	IV-B.1	Full-Schedule Run (IV-B)	Zero late-stage divergence or spikes across full schedule ($>$ 20k steps).	[ ]
	IV-B.2	Validation Check (IV-B)	Held-out validation perplexity satisfies $\Delta\text{PPL} \leq 0.0$.	[ ]
D-Eval	D.1	4-Gate Anti-Regression	Verified non-regression against predecessor version.	[ ]

References

[1] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Budapest, Hungary, 2008, pp. 337–340.

[2] X. Chen *et al.*, "Symbolic discovery of optimization algorithms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 49205–49233.

[3] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, 2019.

[4] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, Stockholm, Sweden, 2018, pp. 1842–1850.

[5] A. Meurer *et al.*, "SymPy: Symbolic computing in Python," *PeerJ Comput. Sci.*, vol. 3, p. e103, Jan. 2017, doi: 10.1717/peerj-cs.103.

[6] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 1737–1746.

[7] P. Zamirai, J. Zhang, C. R. Aberger, and C. De Sa, "Revisiting BFloat16 training," *arXiv preprint arXiv:2010.06192v2*, Mar. 2021.

[8] M. Jamil and X.-S. Yang, "A literature survey of benchmark functions for global optimization problems," *Int. J. Math. Model. Numer. Optim.*, vol. 4, no. 2, pp. 150–194, 2013, doi: 10.1504/IJMMNO.2013.055204.

[9] H. H. Rosenbrock, "An automatic method for finding the greatest or least value of a function," *Comput. J.*, vol. 3, no. 3, pp. 175–184, Oct. 1960, doi: 10.1093/comjnl/3.3.175.

[10] L. A. Rastrigin, *Systems of Extremal Control*. Moscow, Russia: Nauka, 1974.

[11] K. Liang, L. Chen, B. Liu, and Q. Liu, "Cautious optimizers: Improving training with one line of code," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[12] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 8024–8035.

[13] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop Mach. Learn. Program. Lang. (MAPL)*, Phoenix, AZ, USA, Jun. 2019, pp. 10–19, doi: 10.1145/3315508.3329973.

Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.0.0)
Official Engineering Specification — Lumina Moon (cloverx-id)